Module 11 — Project defence and resume interview lab

This module is specific to the three portfolio projects in the preparation plan. Confirm every detail against the actual repository before using it in an interview.

Project walkthrough template

1. User/problem: who needs this and why?

2. Source: where does data come from, and what is the licence/refresh constraint?

3. Grain: what does one source and output row represent?

4. Architecture: ingestion → validation → transform → storage → analysis/UI.

5. Design decisions: why these tools?

6. Quality: tests, validation, reconciliation and rejected records.

7. Failure: one real issue and how it was debugged.

8. Limitations: what is not implemented or not validated?

9. Scale: what changes at 10x volume?

10. Next step: one measurable improvement.

RetailPulse

Explain the retail sales/inventory dataset, source files, transformation pipeline, SQLite warehouse and Streamlit/Plotly dashboard. Be careful: profitability measures should not be presented as validated until the source cost-field mapping is verified. Do not infer transactions from row count if order/transaction grain is not established. Do not claim unsupported customer/returns/geography features.

RetailPulse exercises

• Trace one raw row into a curated table and dashboard metric.

• Explain how you validate key uniqueness and missing values.

• Explain why raw source files are not committed if size limits apply.

• Design a test for category revenue reconciliation.

• Propose a safe incremental refresh if the source publishes new data.

WeatherScope India

Explain daily forecast ingestion from Open-Meteo for five cities, transformation, quality checks, SQLite, reports and Power BI/Streamlit. Row grain is city × forecast date × fetch timestamp. Forecasts are predictions, not observed weather or official warnings. A 50% precipitation probability threshold is a project-defined rule, not an official alert. Do not claim scheduler or production alerting unless implemented.

WeatherScope exercises

• Why is fetch timestamp part of the grain?

• How would you select the latest forecast per city/date?

• How would you compare forecast to actual observed weather if a reliable observation source were added?

• What happens if one city API call fails?

• How do you prevent mixing forecasts from different fetch runs?

TrustLayer

Explain NYC 311 data quality validation, accepted/rejected records, SQLite, pipeline metrics, CSV/JSON summaries, pytest and GitHub Actions. Synthetic test data is for deterministic testing and is not real NYC statistics. A single 1,000-row extract with zero rejects does not prove the entire source is clean. No live dashboard or scheduler is claimed.

TrustLayer exercises

• Design a new validation rule for impossible date chronology.

• Explain why rejected records should retain reason and source row.

• Add a test for duplicate request IDs.

• Explain how API pagination and rate limits would be handled.

• What is the difference between row rejected and pipeline run failed?

Resume defence

For every bullet, prepare:

• what you personally did;

• tools used and why;

• exact input/output;

• evidence (code, tests, screenshot, result);

• limitation;

• what you would improve.

Do not invent business impact, production scale, user count or performance improvement. If you cannot reproduce a metric, remove it or qualify it.

STAR stories

Prepare truthful examples for learning, debugging, ambiguity, feedback, collaboration and a mistake. Situation and task should be brief; actions should focus on your own decisions; result should be concrete but not exaggerated.

External practice and evidence links

Use these to improve the engineering quality of the existing portfolio projects—not to replace the project facts documented above.

1. GitHub Skills — complete Introduction to GitHub and GitHub Actions; apply version control and CI concepts to the repositories.
2. GitHub Actions documentation — add or review automated tests and a clear workflow status.
3. Streamlit Get Started — review app structure, caching and deployment practices for RetailPulse and WeatherScope.
4. Open-Meteo API documentation — verify request parameters, response fields and forecast semantics used by WeatherScope.
5. NYC Open Data — explore the source and data dictionary before proposing new TrustLayer validation rules.
6. Kaggle Datasets — find a separate small dataset for a new extension; do not silently replace the documented source of an existing project.

Evidence task: for each project, add a reproducible setup section, architecture diagram, sample input/output, tests, known limitations and one screenshot captured from the actual running application. Never invent performance metrics or screenshots.